

Milestone 2: Multi-Agent Retrieval-Augmented NDA Review System

Generative and Agentic AI (CSEN 1144) — Spring 2026
German University in Cairo, Department of Computer Science

Generative AI Project Team

April 30, 2026

Contents

1	Executive Summary	3
2	System Architecture	3
2.1	Pipeline Overview	3
2.2	Key Design Decisions	5
3	Retrieval Pipelines	5
3.1	Vector RAG (Chroma + Sentence Transformers)	5
3.2	GraphRAG (Neo4j)	5
3.3	BaseRetriever Interface	6
4	Conversation Agent	6
4.1	Inputs and Outputs	6
4.2	System Prompt and Grounding Rule	6
4.3	Chat Flow	6
4.4	Evidence Extraction and Validation	6
5	Conversation History	7
5.1	ConversationHistory Class	7
5.2	Key Methods	7
6	Updated Playbook	7
6.1	Continuity from Milestone 1	7
6.2	Structure	8
6.3	Per-Hypothesis Entry (Example: H04)	8
6.4	Decision Mapping Logic	8
7	Updated Runtrace Schema	9
7.1	Continuity from Milestone 1	9
7.2	Top-Level Structure	9
7.3	Hypothesis Trace	9
7.4	Metrics Section	9
8	Data Flow and Integration	9
8.1	Contract Input Format	9

8.2 Example Conversation	10
9 Design Constraints	10
10 Testing and Validation	10
10.1 Unit Tests	10
10.2 Validation Metrics	11
11 Implementation Checklist	11
12 Milestone Progression	11
13 Conclusion	11
A Repository Structure	12
B Quick Start	12

1 Executive Summary

Milestone 2 extends the Milestone 1 single-pipeline baseline to a modular, retrieval-augmented NDA review system over ContractNLI. Implemented components include:

- **Dual retrieval pipelines:** Vector RAG (Chroma + `all-MiniLM-L6-v2`) and GraphRAG (Neo4j)
- **Conversation Agent:** Stateful, multi-turn Q&A with evidence grounding
- **Conversation History:** JSON-serializable, session-persistent turn store
- **Updated Playbook:** Hypothesis-level risk classification and action recommendations (JSON)
- **Updated Runtrace Schema:** Structured per-hypothesis decision trace (JSON Schema)

The full 17-hypothesis analysis loop (Hypothesis Analyst + Reviewer agents + Runtrace Formatter) is architectural in this milestone and targeted for Milestone 3.

2 System Architecture

2.1 Pipeline Overview

The Milestone 2 pipeline follows this end-to-end flow:

1. **Input ingestion:** Contract + user prompt + conversation history arrive at the Conversation Agent
2. **Retrieval routing:** User prompt is dispatched to Vector RAG *or* GraphRAG (runtime flag)
3. **Context retrieval:** Top- k precedents fetched from the ContractNLI training split
4. **Evidence grounding:** All cited evidence must be verbatim from the Contract Under Review
5. **Response generation:** Qwen2.5-7B-Instruct generates answer with [EVIDENCE: "..."] tags
6. **Evidence validation:** Tags extracted via regex; each quote verified against contract text
7. **History persistence:** Turn stored with message, response, evidence, precedents, retrieval mode
8. **(M3) Hypothesis loop:** Each of 17 hypotheses processed through Analysis → Playbook → Runtrace

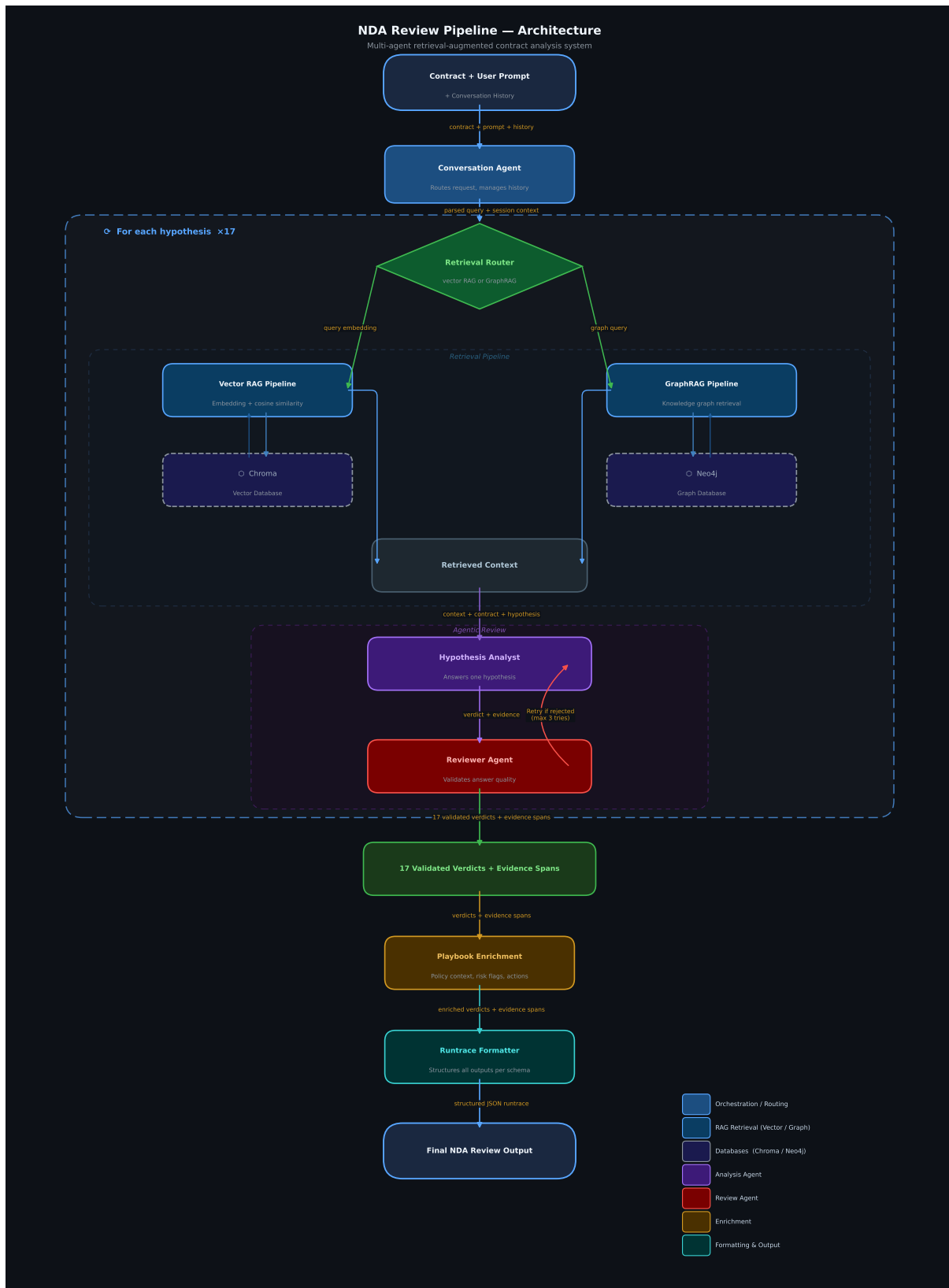


Figure 1: Complete NDA Review Pipeline. Blue: Orchestration. Green: Retrieval. Purple: Analysis (M3). Red: Review (M3). Orange: Enrichment (M3). Dashed box: per-hypothesis loop (Milestone 3).

2.2 Key Design Decisions

Conversation Agent as Entry Point. The Conversation Agent acts as the primary orchestrator, enabling natural user interaction, context reuse across turns, and flexible retrieval routing via a single runtime parameter.

Dual Retrieval Modes. Vector RAG (Chroma + cosine similarity) is fast and effective for semantic matching. GraphRAG (Neo4j Cypher) captures structured clause relationships and logical dependencies. Both implement a common `BaseRetriever` interface, making them interchangeable.

Evidence Grounding Rule. Retrieved precedents are reasoning context only. Every cited evidence item must be a verbatim quote from the Contract Under Review, enforced via system prompt embedding, [EVIDENCE: "..."] tag schema, and post-hoc verification.

3 Retrieval Pipelines

3.1 Vector RAG (Chroma + Sentence Transformers)

Documents from the training split are chunked by spans, embedded with `all-MiniLM-L6-v2`, and stored in a Chroma collection. At query time:

```
def retrieve_vector(query, k=5):
    query_embedding = model.encode(query)
    results = collection.query(
        query_embeddings=[query_embedding], n_results=k,
        include=["documents", "metadatas", "distances"]
    )
    return [{
        "label": meta.get("doc_id"),
        "text": doc,
        "score": 1.0 - dist,    # cosine distance -> similarity
        "chunk_id": chunk_id
    } for doc, meta, dist, chunk_id in zip(...)]
```

Listing 1: Vector RAG Query

Chunks respect original document spans to preserve contextual boundaries.

3.2 GraphRAG (Neo4j)

The training split is transformed into a property graph. Node types: `Clause`, `Obligation`, `Condition`, `Party`, `Exception`. Relationships: `HAS_OBLIGATION`, `SUBJECT_TO`, `CONTAINS_EXCEPTION`, `RELATED_TO`, `CONFLICTS_WITH`.

At query time, key entities are extracted from the user prompt and a Cypher query retrieves related clauses:

```
query = """
MATCH (c:Clause)-[:HAS_OBLIGATION]->(o:Obligation)
WHERE toLower(o.text) CONTAINS toLower($keyword)
RETURN DISTINCT c.text AS clause_text, c.doc_id,
count(o) AS obligation_count
LIMIT $k
"""
results = execute_cypher(query, {"keyword": keyword, "k": 5})
```

Listing 2: GraphRAG Retrieval

3.3 BaseRetriever Interface

Both pipelines implement a common interface, allowing the Conversation Agent to swap modes via a single config parameter:

```
class BaseRetriever(ABC):
    @property
    def mode(self) -> str: ...          # "vector" or "graphrag"

    @abstractmethod
    def retrieve(self, query: str, k: int = 5) -> List[Dict]: ...
    # Returns: [{"label", "text", "score", "chunk_id"}, ...]
```

Listing 3: Retriever Interface

4 Conversation Agent

4.1 Inputs and Outputs

The agent accepts a contract dict ({id, text, spans}), a user_prompt, and a ConversationHistory object. It returns a response string, extracted evidence items, retrieved precedents, retrieval_mode, and token usage.

4.2 System Prompt and Grounding Rule

```
You are an expert legal NDA analyst. Answer questions about
the contract below.

CONTRACT UNDER REVIEW (ID: {contract_id})
{contract_text}

EVIDENCE GROUNDING RULE (mandatory):
    Precedents in user messages are reasoning context ONLY.
    Every cited evidence MUST be verbatim from the contract above.
    Tag every quote: [EVIDENCE: "exact quote"]
    Never fabricate, paraphrase, or cite precedents as evidence.

RESPONSE STRUCTURE:
1. Direct answer    2. Contract evidence    3. Risk note
```

Listing 4: System Prompt (condensed)

4.3 Chat Flow

1. Retrieve top- k precedents via `retriever.retrieve(user_prompt)`
2. Build messages: system prompt + history + augmented user message
3. Call Qwen2.5-7B-Instruct via HuggingFace Serverless Inference API
4. Extract evidence via regex: `r'[EVIDENCE:\s*"((?:[\\]|\)|\.)*)"\s*']`
5. Verify each quote (exact match → case-insensitive fallback → mark unverified)
6. Persist turn to history

4.4 Evidence Extraction and Validation

```
@staticmethod
def _extract_evidence(response_text, contract_text):
```

```

pattern = re.compile(r'\[EVIDENCE:\s*"((?:[^\"]|\\.)*)"')
seen, evidence = set(), []
for m in pattern.finditer(response_text):
    quote = m.group(1).strip()
    if quote in seen: continue
    seen.add(quote)
    start = contract_text.find(quote) # exact match
    if start == -1:
        start = contract_text.lower().find(quote.lower()) # CI fallback
    evidence.append({
        "quote": quote,
        "char_start": start, "char_end": start + len(quote),
        "verified": start != -1
    })
return evidence

```

Listing 5: Evidence Extraction

Retrieved precedents are clearly separated in the user message with a **RETRIEVED PRECEDENTS** --- reasoning context only header to ensure the LLM does not cite them as contract evidence.

5 Conversation History

5.1 ConversationHistory Class

Each session has a unique UUID-derived `session_id` and an ordered list of turns. Each turn stores:

Field	Description	Type
<code>turn_id</code>	Sequential index	int
<code>timestamp</code>	ISO 8601 UTC	str
<code>user</code>	Original (un-augmented) prompt	str
<code>assistant</code>	LLM response with evidence tags	str
<code>evidence</code>	Extracted + verified quotes	list[dict]
<code>precedents</code>	Retrieved training examples	list[dict]
<code>retrieval_mode</code>	"vector" or "graphrag"	str

5.2 Key Methods

`add_turn()` appends a new turn. `to_anthropic_messages()` exports history as alternating `user/assistant` dicts for the inference API. `save(path)` / `load(path)` serialize to/from JSON for cross-session persistence.

6 Updated Playbook

6.1 Continuity from Milestone 1

The playbook is **unchanged from Milestone 1**. After finalizing the Milestone 2 multi-agent architecture, we determined that the two-agent design — a Hypothesis Analyst that processes all 17 hypotheses and a Reviewer that validates each decision and may force the Analyst to revise its prediction — does not require any structural changes to the playbook. The risk classification logic, severity mappings, action types, and per-hypothesis entries defined in Milestone 1 remain fully compatible with the new agentic flow. The playbook file is provided as `playbook.json`.

6.2 Structure

The playbook is a schema-validated JSON file mapping each of the 17 ContractNLI hypotheses to risk classification and recommended action.

```
{
  "playbook_id": "contractnli_nda_minimal",
  "version": "1.1",
  "global_defaults": {
    "label_to_status": {
      "ENTAILED": "satisfied",
      "CONTRADICTED": "conflict",
      "NOT_MENTIONED": "missing"
    },
    "label_to_default_decision": {
      "ENTAILED": { "severity": "LOW", "action": "ACCEPT" },
      "CONTRADICTED": { "severity": "HIGH", "action": "ESCALATE" },
      "NOT_MENTIONED": { "severity": "MEDIUM", "action": "CLARIFY" }
    }
  },
  "criticality_tiers": {
    "P0": "Deal-critical / baseline NDA hygiene",
    "P1": "Important / commonly negotiated",
    "P2": "Optional or context-dependent"
  }
}
```

Listing 6: Playbook Top-Level

6.3 Per-Hypothesis Entry (Example: H04)

```
{
  "hypothesis_id": "H04",
  "title": "Purpose limitation",
  "hypothesis_text": "Receiving Party shall not use any Confidential  
Information for any purpose other than the  
purposes stated in Agreement.",
  "criticality": "P0",
  "overrides": {
    "NOT_MENTIONED": { "severity": "HIGH", "action": "CLARIFY" }
  },
  "rationale_templates": {
    "satisfied": "NDA restricts use to stated purpose(s). ...",
    "conflict": "NDA allows use beyond stated purpose(s). ...",
    "missing": "NDA does not restrict use to stated purpose(s). ..."
  }
}
```

Listing 7: H04 Purpose Limitation

6.4 Decision Mapping Logic

For each hypothesis: (1) generate NLI label; (2) check playbook overrides; (3) fall back to global defaults; (4) fill rationale template. The four possible actions are **ACCEPT**, **CLARIFY**, **NEGOTIATE**, and **ESCALATE**. In Milestone 3, the Reviewer agent may reject a decision and trigger a re-run of step (1); the playbook mapping in steps (2)–(4) is applied identically to any revised label.

7 Updated Runtrace Schema

7.1 Continuity from Milestone 1

The runtrace schema is also **unchanged from Milestone 1**. The Milestone 2 multi-agent design introduces a Reviewer agent that can force the Hypothesis Analyst to revise predictions, but this interaction is internal to the pipeline — the final output written to the runtrace is identical in structure to what Milestone 1 defined. Each hypothesis still produces one trace entry with the same decision object, evidence, and validation fields. The runtrace file is provided as `runtrace_schema.json`.

7.2 Top-Level Structure

```
{
  "schema_version": "1.0-ms1",
  "run": {
    "run_id": "run_id", "started_at": "started_at", "ended_at": "ended_at",
    "framework": "framework", "parameters": {...} },
  "contract": { "contract_id": "contract_id", "source_name": "source_name", "chunks": [...] },
  "playbook": { "playbook_id": "playbook_id", "version": "version" },
  "hypothesis_traces": [ /* 17 entries, one per hypothesis */ ],
  "metrics": { "contract_accuracy": "contract_accuracy", "groundedness_rate": "groundedness_rate", ... },
  "run_validations": [ /* pass/fail checks */ ]
}
```

Listing 8: Runtrace Top Level

7.3 Hypothesis Trace

Each trace contains:

- **Metadata:** `hypothesis_id`, `hypothesis_text`, `gold_label`, `latency_ms`
- **Steps:** `load_inputs` → `nli_infer` → `evidence_map` → `playbook_map` → `consistency_check` → `format_response`
- **Decision:** predicted label, confidence, supporting evidence quotes with character offsets, risk severity, criticality tier, recommended action
- **Validations:** evidence grounding check, quote integrity check, label consistency check

Note that if the Reviewer agent triggers a revision in Milestone 3, only the final accepted decision is written to the trace. Intermediate rejected predictions are not persisted in the runtrace, keeping the schema identical to its Milestone 1 definition.

7.4 Metrics Section

Key metrics aggregated across all 17 hypotheses:

Metric	Description
<code>contract_accuracy</code>	Fraction of hypotheses with correct label
<code>groundedness_rate</code>	Fraction of evidence from contract (target: 1.0)
<code>quote_integrity_rate</code>	Fraction of quotes located verbatim
<code>contract_latency_ms</code>	Total processing time

8 Data Flow and Integration

8.1 Contract Input Format

```
contract = {
    "id": "contract_001",
    "text": "Full contract text...",
    "spans": [{"char_start": 0, "char_end": 500, "section": " 1 "}]
}
```

8.2 Example Conversation

```
retriever = VectorRetriever(model_name="all-MiniLM-L6-v2")
agent = ConversationAgent(retriever, hf_token=os.getenv("HF_TOKEN"),
                        model="Qwen/Qwen2.5-7B-Instruct")
history = ConversationHistory()

result = agent.chat(contract,
    "Does this NDA allow employee access to confidential info?",
    history)
# result["response"] contains [EVIDENCE: "..."] tagged answer
# result["evidence"] lists verified quotes with char offsets

# Follow-up
result2 = agent.chat(contract, "What about third-party access?", history)

# Persist
history.save("conversation_001.json")
```

Listing 9: End-to-End Usage

9 Design Constraints

1. **Dataset continuity:** Same evaluation split (contracts) and training split (retrieval corpus) as Milestone 1
2. **Base model family:** Qwen2.5-7B-Instruct (Qwen family, consistent with M1)
3. **Evidence grounding:** All cited evidence verbatim from contract; external retrieval is reasoning context only
4. **Conversation history:** Multi-turn interaction with JSON-serializable persistence
5. **Dual retrieval:** Both Vector RAG and GraphRAG implemented; selection via runtime flag

Grounding is enforced at three levels: (1) system prompt instruction; (2) [EVIDENCE: "..."] tag requirement; (3) post-generation verification with character-offset matching.

10 Testing and Validation

10.1 Unit Tests

Key tests cover evidence extraction (exact match, case-insensitive fallback, not-found), conversation history (add turn, JSON persistence roundtrip), and retriever interface compliance.

```
def test_exact_match():
    response = '[EVIDENCE: "must destroy information"]'
    contract = "All data must destroy information upon termination."
    ev = ConversationAgent._extract_evidence(response, contract)
    assert ev[0]["verified"] is True and ev[0]["char_start"] == 17

def test_not_found():
    response = '[EVIDENCE: "nonexistent phrase"]'
```

```
ev = ConversationAgent._extract_evidence(response, "Some_text.")
assert ev[0]["verified"] is False
```

Listing 10: Evidence Extraction Tests (condensed)

10.2 Validation Metrics

Metric	Target
Evidence Grounding Rate	1.0 (all evidence from contract)
Quote Integrity Rate	≥ 0.95 (quotes found verbatim)
Evidence Coverage	≥ 1 evidence item per response
Latency	$< 5s$ per turn (HF Serverless)

11 Implementation Checklist

Component	Status	Location
Vector RAG (Chroma + embeddings)		vector_rag_setup.ipynb
GraphRAG (Neo4j)		graphrag_pipeline.ipynb
BaseRetriever interface		retrieval/base.py
ConversationAgent		conversation_agent.py
Evidence extraction + verification		conversation_agent.py
ConversationHistory + persistence		history.py
Updated playbook (JSON, 17 entries)		schemas/playbook.JSON
Updated runtrace schema (JSON)		schemas/runtrace_schema.json
Architecture diagram (PDF + JSON)		docs/
Hypothesis Analyst Agent	×	Milestone 3
Reviewer Agent (retry logic)	×	Milestone 3
Playbook enrichment integration	×	Milestone 3
Runtrace Formatter	×	Milestone 3
17-hypothesis orchestration	×	Milestone 3

12 Milestone Progression

	Milestone 1	Milestone 2 (this)	Milestone 3 (planned)
Scope	Single inference pipeline	Modular RAG + conversation	Full agentic system
Input	Contract + hypothesis text	Contract + prompt + history	All 17 hypotheses
Output	Per-hypothesis NLI labels	Free-form response + evidence	Structured runtrace
Architecture	Monolithic	Modular, pluggable retrieval	Multi-agent orchestration
Interaction	Batch	Multi-turn interactive	Batch + interactive

13 Conclusion

Milestone 2 establishes a fully functional, modular retrieval-augmented system for conversational NDA review. Both retrieval modes are implemented and interchangeable, the evidence

grounding rule is enforced end-to-end, and multi-turn conversation history is preserved across sessions. The updated playbook and runtrace schema are defined and ready for integration with the agentic hypothesis analysis planned in Milestone 3.

A Repository Structure

```
contractnli-ms2/
  src/
    conversation_agent.py
    history.py
    retrieval/
      base.py
      vector_rag.py
      graphrag.py
  notebooks/
    vector_rag_setup.ipynb
    graphrag_pipeline.ipynb
  schemas/
    playbook.JSON
    playbook_schema.json
    runtrace.json
    runtrace_schema.json
  data/          # ContractNLI evaluation split
  tests/
  docs/
    ARCHITECTURE.md and nda_pipeline_diagram.pdf
  requirements.txt
```

B Quick Start

```
git clone <repo-url> && cd contractnli-ms2
git checkout ms-2
python3.10 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
export HF_TOKEN="..." NEO4J_URI="bolt://localhost:7687" NEO4J_PASSWORD="..."
```

```
from src.conversation_agent import ConversationAgent
from src.retrieval.vector_rag import VectorRetriever
from src.history import ConversationHistory
import json

with open("data/contract_001.json") as f:
    contract = json.load(f)

retriever = VectorRetriever(model_name="all-MiniLM-L6-v2")
agent = ConversationAgent(retriever, hf_token="...",
                        model="Qwen/Qwen2.5-7B-Instruct")
history = ConversationHistory()
result = agent.chat(contract, "What are confidentiality obligations?", history)
print(result["response"])
history.save("conversation_001.json")
```